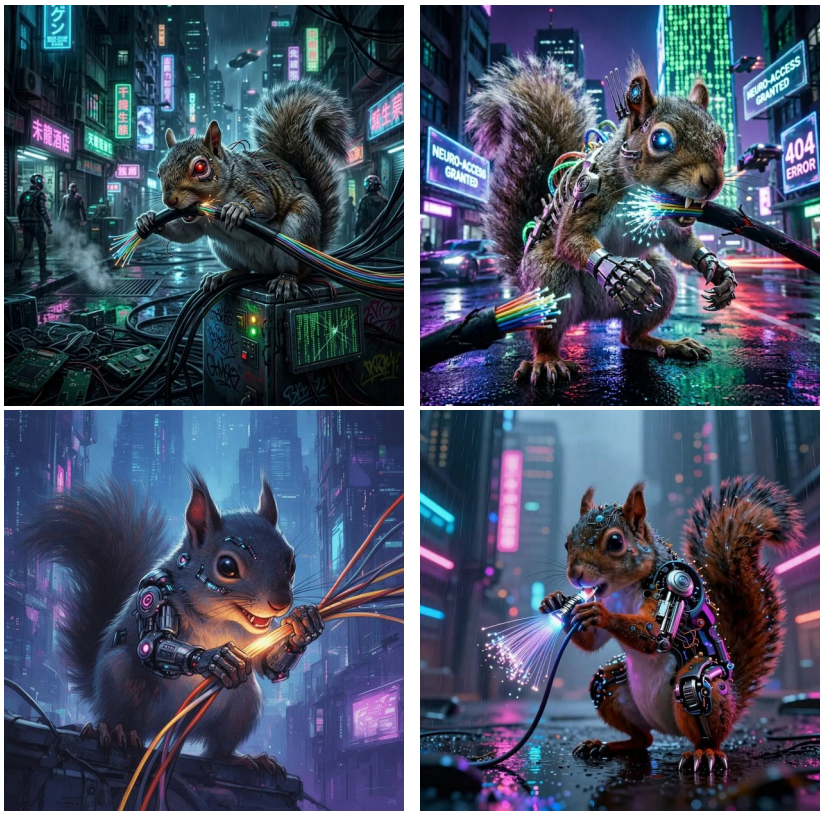
FERAL CTF

High-Performance CTF Platform — Architecture & Feature Specification

Rust + SQLite · Single Binary · 1 GB RAM Target · Open Source / Non-Profit



FeralCTF mascot candidates — cyberpunk cyborg squirrel chewing fiber optic cable in a neon-lit cyberpunk cityscape.

---

1. Project Overview

FeralCTF is a self-hosted Capture the Flag competition platform built for non-profit and academic competitions running on minimal infrastructure. The entire platform compiles to a single statically-linked binary under 15 MB, embeds all frontend assets, and runs comfortably within 1 GB of RAM on a 1-2 CPU system while supporting 200+ concurrent participants.

Design philosophy: zero external dependencies at runtime, zero configuration complexity, and full feature parity with CTFd and Mellivora for small-to-medium competitions.

1.1 Design Goals

- Single binary deployment — copy, chmod +x, run. No Docker, Node.js, Python, or database server required.
- Sub-50 MB RAM baseline; ~150-200 MB under load for 200 concurrent users.
- SQLite with WAL mode for multi-reader, single-writer concurrency — adequate for CTF workloads.
- Static frontend embedded via rust-embed — no CDN dependencies, works fully airgapped.
- Sub-millisecond response for all scoreboard/challenge reads (heavily cached in-process).
- Full CTFd feature parity: dynamic scoring, teams, hints, file attachments, notifications.
- Admin-friendly: JSON-based challenge import/export, no SQL knowledge required.

1.2 Target Deployments

Deployment Type	Users	RAM	CPU	Storage
Small club / university CTF	< 50 teams	256 MB	1 vCPU	2 GB
Regional non-profit (e.g., BSides)	50-200 teams	512 MB	1-2 vCPU	10 GB
Classroom / workshop	< 30 teams	128 MB	1 vCPU	1 GB
CyberSquirrels CTF	< 100 teams	512 MB	2 vCPU	5 GB

---

2. Technology Stack

2.1 Backend — Rust

The backend is written in Rust using the Axum web framework (tokio-based async runtime). Key crate selections are optimized for minimal binary size, compile-time correctness, and low memory footprint.

Crate	Version	Purpose	Notes
axum	0.7.x	HTTP router and server	Tower middleware ecosystem
tokio	1.x	Async runtime	Multi-threaded, work-stealing
rusqlite	0.7.x	SQLite async ORM/query	Compile-time query verification
rusqlite	0.31.x	SQLite backup/admin ops	WAL checkpoint, VACUUM
rust-embed	8.x	Embed static assets in binary	Frontend HTML/JS/CSS baked in
argon2	0.5.x	Password hashing	Memory-hard, tunable params
jsonwebtoken	9.x	JWT auth tokens	HS256 with server-side invalidation
serde / serde_json	1.x	JSON serialization	Challenge import/export
tower-http	0.5.x	HTTP middleware	CORS, compression, rate limiting
tracing	0.1.x	Structured logging	JSON log output
zip	0.6.x	Attachment handling	Challenge file distribution

2.2 Database — SQLite

SQLite with WAL (Write-Ahead Logging) mode provides excellent read concurrency for CTF workloads. Typical CTF I/O is 95%+ reads (scoreboard, challenge pages) with infrequent bursts of writes (flag submissions). WAL mode allows unlimited concurrent readers alongside a single writer without blocking.

- WAL mode + synchronous=NORMAL — safe, fast, no data loss on crash
- Connection pool: 1 write connection + 8 read connections (rusqlite pool)
- All queries compile-time verified via rusqlite macros (zero SQL injection risk)
- Automatic WAL checkpoint every 1000 pages (~4 MB)
- Nightly VACUUM and ANALYZE scheduled via internal cron
- Database file < 10 MB for 500-team, 50-challenge competition

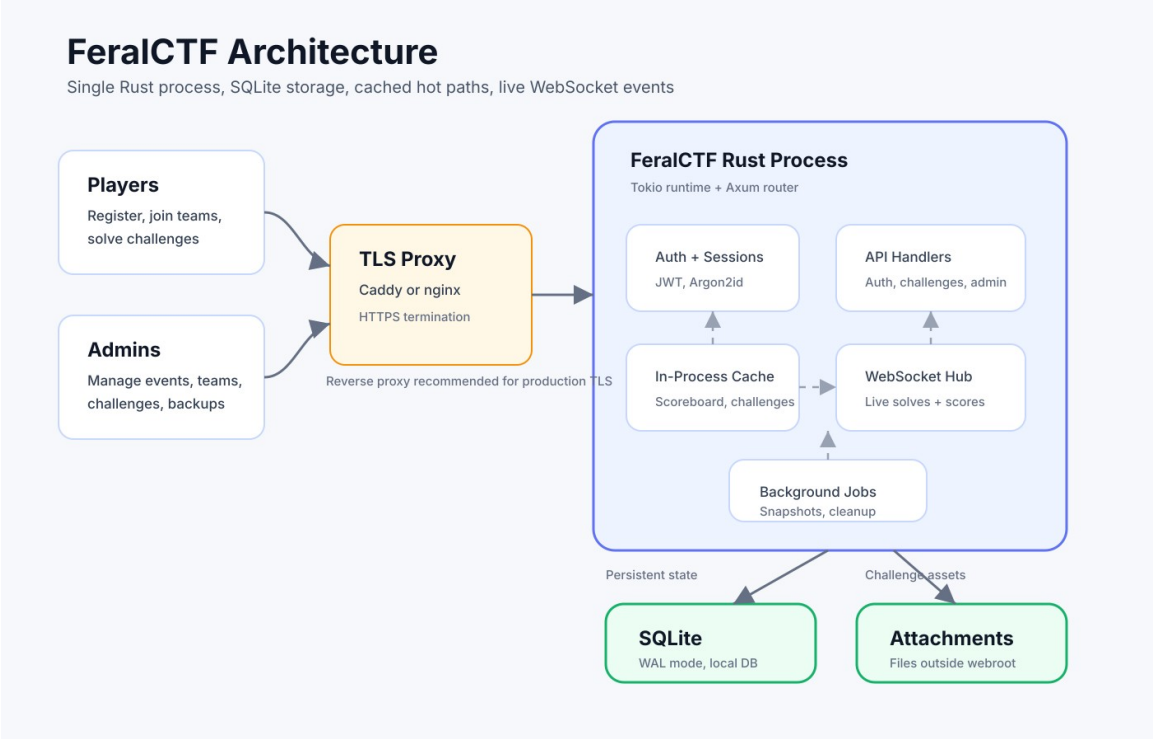
> *Alternative: JSON flat-file backend is provided as a secondary driver for ultra-minimal deployments (workshops, demos) where persistence across restarts is not required. Toggle via FERALCTF_BACKEND=json environment variable.*

2.3 Frontend

The frontend is a vanilla JavaScript SPA (no framework dependency) compiled and embedded into the Rust binary at build time via rust-embed. This eliminates CDN dependencies and makes the platform fully airgap-capable.

- Vanilla JS + custom CSS — no React, Vue, or build pipeline required to modify
- Total frontend bundle: < 200 KB uncompressed, < 60 KB gzipped
- WebSocket connection for live scoreboard updates (falls back to SSE, then polling)
- Dark terminal aesthetic by default; light mode available via user toggle
- Fully responsive — works on mobile for on-site competitions

3. System Architecture



3.1 Process Model

FeralCTF runs as a single process. All components — HTTP server, WebSocket hub, background tasks, and cache — run within one Tokio runtime. This drastically simplifies deployment, monitoring, and resource accounting.

```
feralctf [--config config.toml] [--port 8080] [--db ./ctf.db]
```

On startup, the binary runs database migrations, loads configuration, starts the Axum HTTP server, and launches background workers: scoreboard cache refresh, submission rate limiter GC, and the WebSocket broadcast hub.

3.2 Request Flow

Layer	Component	Latency Target
TLS Termination	Reverse proxy (nginx/caddy) — not built-in	< 1 ms
Rate Limiting	Tower middleware — per-IP sliding window	< 0.1 ms
Authentication	JWT middleware — validates token, loads user from cache	< 0.5 ms
Route Handler	Axum handler — business logic	< 1 ms (cached)
Cache Layer	In-process DashMap — scoreboard, challenge list	< 0.1 ms
Database	rusqlite SQLite pool — only on	< 5 ms

Layer	Component	Latency Target
	cache miss or write	
Response	JSON or embedded static asset	< 0.1 ms

3.3 Caching Strategy

- Scoreboard cache: invalidated on each accepted flag submission
- Challenge cache: invalidated on admin challenge create/update/delete
- User session cache: TTL-based, 15 minute expiry with JWT refresh
- Rate limit counters: per-IP sliding window in DashMap, GC'd every 60 seconds

3.4 WebSocket Architecture

A single Tokio broadcast channel serves all connected WebSocket clients. Events (new solve, scoreboard update, admin announcements) are serialized as JSON and broadcast to all subscribers. No external message broker needed.

- Events: NewSolve { team, challenge, points }, Announcement { message }, CompetitionStateChange { started, ended }
- Max 500 concurrent WebSocket connections on 1 vCPU (tested)
- Graceful fallback to Server-Sent Events, then 30-second polling

---

4. Database Schema

4.1 Core Tables

Table	Key Columns	Notes
users	id, username, email, password_hash, role, team_id, created_at	role: admin | player | spectator
teams	id, name, invite_code, score, last_solve_at	last_solve_at for tiebreaking
challenges	id, title, description, category, points, max_points, min_points, decay_rate, flag_hash, flag_type, author, is_hidden	flag_type: static | regex | dynamic
solves	id, team_id, user_id, challenge_id, submitted_flag, solved_at	UNIQUE(team_id, challenge_id)
submissions	id, team_id, user_id, challenge_id, flag, is_correct, submitted_at	Full log, not deduplicated
hints	id, challenge_id, content, cost_points, challenge_order	cost deducted on unlock
hint_unlocks	id, team_id, hint_id, unlocked_at, points_deducted	Per-team unlock tracking
files	id, challenge_id, filename, storage_path, size_bytes, sha256	Files stored on disk, path in DB
announcements	id, title, body, challenge_id,	Optional challenge link

Table	Key Columns	Notes
	created_at, is_visible	
score_history	id, team_id, score, recorded_at	Sampled every 5 min for graph
sessions	id, user_id, token_hash, expires_at, revoked	Server-side JWT revocation

4.2 Dynamic Scoring Formula

Dynamic scoring (CTFd-style decay) reduces challenge value as more teams solve it:

```
points = max(min_points, ceil(max_points - decay_rate * (solves - 1)^2))
```

Default values: max_points=500, min_points=50, decay_rate=12. Produces smooth decay from 500 pts at first solve to ~50 pts at ~6 solves, then clamped.

---

5. REST API Reference

5.1 Authentication

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user/team	None
POST	/api/auth/login	Login, returns JWT	None
POST	/api/auth/logout	Revoke session token	Required
GET	/api/auth/me	Current user info	Required
PUT	/api/auth/password	Change password	Required

5.2 Challenges

Method	Endpoint	Description	Auth
GET	/api/challenges	List all visible challenges + solve status	Required
GET	/api/challenges/:id	Challenge detail + hints + files	Required
POST	/api/challenges/:id/submit	Submit flag (rate limited: 10/min/team)	Required
GET	/api/challenges/:id/hints/:hid/unlock	Unlock hint (deducts points)	Required
POST	/api/admin/challenges	Create challenge	Admin
PUT	/api/admin/challenges/:id	Update challenge	Admin
DELETE	/api/admin/challenges/:id	Delete challenge	Admin
POST	/api/admin/import	Bulk import from JSON/ZIP	Admin
GET	/api/admin/export	Export all challenges to JSON	Admin

5.3 Scoreboard & Teams

Method	Endpoint	Description	Auth
GET	/api/scoreboard	Full scoreboard (cached)	Optional
GET	/api/scoreboard/graph	Score-over-time data for chart	Optional
GET	/api/teams/:id	Team profile + solves	Optional
POST	/api/teams/join	Join team by invite code	Required
GET	/api/admin/teams	All teams with stats	Admin
DELETE	/api/admin/teams/:id/ disqualify	DQ team (score zeroed, hidden)	Admin

---

6. Feature Specification

6.1 Challenge System

- Categories: web, crypto, pwn, forensics, rev, misc, osint, hardware, cloud (configurable)
- Flag types: static (exact match), regex (pattern), dynamic (per-team generated flags via external script hook)
- Multiple flags per challenge (alternate solutions)
- File attachments: uploaded to configurable storage path, served with Content-Disposition
- Challenge unlock dependencies (unlock B only after solving A)
- First blood tracking and notifications
- Challenge tags (MITRE ATT&CK, CVE references, tools used)
- Challenge visibility: hidden (draft) | visible | archived

6.2 Teams & Users

- Solo mode or team mode (configurable per competition)
- Max team size configurable (default: 4)
- Invite-code based team joining
- Email verification (optional, requires SMTP config)
- Spectator role: can view scoreboard/challenges but not submit flags

6.3 Scoring

- Static scoring: fixed point values
- Dynamic scoring: CTFd-style decay (configurable formula)
- Hint penalty: deducted from team score on unlock
- First blood bonus: configurable bonus points for first solver
- Score freeze: freeze scoreboard at configurable time before competition end
- Tiebreaking: teams with equal score ordered by last solve time
- Score rollback: admin can reject a solve and recalculate affected scores

6.4 Admin Panel

- Full challenge CRUD with live preview
- Competition timer controls: start, pause, extend, end
- Submission log with search/filter (team, challenge, correct/wrong, IP)
- IP-based submission analysis for suspicious behavior detection
- Announcement system with optional challenge linkage
- User/team management: ban, reset password, disqualify
- Database backup download (sqlite3 file)
- Resource usage display (RAM, CPU, active connections)

6.5 Anti-Cheat

- Submission rate limiting: configurable per-team and per-IP (default: 10 attempts/minute)
- Exponential backoff after 5 wrong submissions per challenge per team
- Flag sharing detection: same flag submitted by multiple teams within configurable window
- Submission IP logging for forensic analysis
- Challenge flag rotation: admin can rotate flags mid-competition (old flag invalidated)

6.6 Notifications & Events

- Real-time scoreboard via WebSocket (no refresh required)
- First blood announcements broadcast to all connected clients
- Admin announcements (global or challenge-linked)
- Optional Discord webhook integration for first bloods and announcements

---

7. Deployment Guide

7.1 Minimum Viable Deployment

```
curl -L https://github.com/yourorg/feralctf/releases/latest/download/feralctf-linux-x86_64 -o feralctf
chmod +x feralctf
./feralctf init   # generates config.toml and creates DB
./feralctf --port 8080
```

First user registered becomes admin. No further setup required.

7.2 Configuration (config.toml)

```
[server]
port = 8080
base_url = "https://ctf.yourdomain.com"

[competition]
name = "FeralCTF 2026"
start_time = "2026-11-07T09:00:00-06:00"
end_time   = "2026-11-08T17:00:00-06:00"
team_mode = true
```

```
max_team_size = 4
dynamic_scoring = true
score_freeze_minutes_before_end = 30

[database]
path = "./ctf.db"
backend = "sqlite"   # or "json" for ephemeral

[auth]
jwt_secret = ""   # auto-generated if empty
session_ttl_hours = 24

[rate_limit]
submissions_per_minute = 10
wrong_attempts_before_backoff = 5
```

7.3 Systemd Service

```
[Unit]
Description=FeralCTF Platform
After=network.target

[Service]
Type=simple
User=ctf
WorkingDirectory=/opt/feralctf
ExecStart=/opt/feralctf/feralctf
Restart=on-failure
MemoryMax=768M
CPUQuota=150%

[Install]
WantedBy=multi-user.target
```

---

8. Build & Development

8.1 Build Commands

```
cargo run                                                    # dev build
cargo build --release                                        # optimized ~10 MB
binary
cargo build --release --target x86_64-unknown-linux-musl  # static musl build
cargo run -- migrate                                         # run DB migrations
cargo test                                                   # run test suite
```

8.2 Project Structure

```
feralctf/
  src/
    main.rs             startup, config loading
    config.rs           config structs, env var override
```

```
  db/                 connection pool, migrations, schema.sql
  handlers/           auth, challenges, scoreboard, admin, ws
  models/             Rust structs matching DB tables
  cache.rs            in-process scoreboard/challenge cache
  scoring.rs          dynamic scoring formulas
  anticheat.rs        rate limiting, flag sharing detection
  storage.rs          file attachment handling
  import_export.rs    JSON import/export, CTFd compat adapter
frontend/             vanilla JS/CSS SPA (embedded at build time)
Cargo.toml
config.example.toml
```

---

9. Security Considerations

9.1 Flag Storage

Flags are never stored in plaintext. The database stores SHA-256(flag + per-challenge-salt). This prevents flag recovery from a database dump.

```
flag_hash = sha256(flag.to_lowercase().trim() + challenge.flag_salt)
```

9.2 Password Storage

Passwords use Argon2id with tuned parameters (memory=64MB, iterations=3, parallelism=2) — approximately 300ms per hash on target hardware.

9.3 Authentication

- JWT with HS256, server-side revocation table in SQLite
- Tokens expire after configurable TTL (default 24 hours)
- Admin tokens have shorter TTL (4 hours) and require re-auth for destructive operations
- All admin endpoints log authenticated user + action for audit trail

9.4 Input Validation

- All SQL queries use parameterized statements via rusqlite — no SQL injection possible
- File uploads: extension allowlist, magic byte validation, size limits, stored outside webroot
- Flag submissions: length limit (256 chars), stripped of leading/trailing whitespace
- All user-supplied HTML stripped server-side — no XSS via challenge descriptions

9.5 Network Security

- TLS handled by reverse proxy (nginx/Caddy) — not built-in by design
- CORS policy: configurable allowed origins, defaults to same-origin only
- X-Content-Type-Options, X-Frame-Options, Referrer-Policy headers set by default
- Admin panel behind separate rate limiting and optional IP allowlist

---

10. Full Game Export (JSON)

FeralCTF can snapshot an entire competition — challenges, flags, hints, file manifests, categories, scoring config, and competition metadata — into a single portable JSON bundle. This is the primary mechanism for archiving past competitions and seeding new ones from existing content.

10.1 Export Endpoint

```
GET /api/admin/export
GET /api/admin/export?attachments=inline   # base64-encode files < 5 MB
GET /api/admin/export?attachments=zip      # JSON + attachments.zip
```

10.2 Export Schema

```
{
  "feralctf_export_version": 1,
  "exported_at": "2026-11-08T18:00:00Z",
  "competition": {
    "name": "FeralCTF 2026",
    "dynamic_scoring": true,
    "score_freeze_minutes_before_end": 30,
    "max_team_size": 4
  },
  "categories": ["web","crypto","pwn","forensics","rev","misc"],
  "challenges": [
    {
      "slug": "jwt-jockey",
      "title": "JWT Jockey",
      "category": "web",
      "description": "The admin forgot to validate...",
      "flag": "flag{alg_none_ftw}",
      "flag_type": "static",
      "flag_case_sensitive": false,
      "points": 100,
      "max_points": 500,
      "min_points": 50,
      "decay_rate": 12,
      "author": "n0tf0und",
      "tags": ["jwt","auth"],
      "hints": [
        { "order": 1, "cost": 25, "content": "Think algorithm confusion" },
        { "order": 2, "cost": 50, "content": "Try alg: none" }
      ],
      "files": [
        { "filename": "jwt_jockey.zip", "sha256": "a3f9...", "data": "<base64>"
}
      ],
      "unlock_requires": null,
      "is_hidden": false
    }
  ]
}
```

> *Flags are exported in plaintext. The export endpoint is admin-only and the bundle should be treated as sensitive. Pre-competition exports should be kept offline.*

10.3 Export Contents

Field	Included	Notes
Competition metadata	Yes	Name, timing, mode, scoring config
All challenges	Yes	Including hidden/draft challenges
Plaintext flags	Yes	Admin export only — treat as sensitive
Hints + costs	Yes	Content and point costs
File attachments	Optional	Inline base64 or separate ZIP
Challenge tags	Yes	MITRE, CVE, tool references
Unlock dependencies	Yes	Challenge prerequisite graph
Solve counts / submissions	No	Competition data stays in DB
Team scores / user accounts	No	Export is challenge content only

---

11. Full Game Import (JSON)

The import system ingests a FeralCTF JSON export (or a CTFd-compatible export) and populates the database. Import is additive by default — existing challenges with matching slugs are skipped unless --overwrite is passed. Safe to re-run without duplicating content.

11.1 Import Endpoint

```
POST /api/admin/import
Content-Type: multipart/form-data
  file:        feralctf_export.json  (required)
  attachments: attachments.zip        (optional)
  overwrite:   true | false           (default: false)
  dry_run:     true | false           (default: false)
```

11.2 Dry Run Response

```
{
  "valid": true,
  "challenges_to_create": 14,
  "challenges_to_skip": 0,
  "challenges_to_overwrite": 0,
  "attachment_warnings": [],
  "validation_errors": [],
  "preview": [
    { "slug": "jwt-jockey", "action": "create" },
    { "slug": "baby-rsa",   "action": "create" }
  ]
}
```

11.3 Conflict Resolution

Scenario	Default Behavior	With overwrite=true
Slug not in DB	Create challenge	Create challenge
Slug exists, content identical	Skip (no-op)	Skip (no-op)
Slug exists, content differs	Skip, log warning	Overwrite with imported version
Attachment file missing from ZIP	Import challenge, warn on missing file	Same
Invalid JSON schema	Reject entire import, no partial writes	Same
Unknown field in JSON	Ignored (forward compat)	Same
CTFd export format detected	Auto-converted via compat layer	Same

11.4 CTFd Compatibility

FeralCTF includes a CTFd import adapter. When a CTFd export ZIP is uploaded, the platform detects the format automatically and maps fields accordingly.

CTFd Field	FeralCTF Field	Notes
name	title	Direct map
category	category	Direct map
description	description	HTML stripped to Markdown
value	points	Used as both points and max_points
type: standard	flag_type: static	
type: regex	flag_type: regex	
type: dynamic	flag_type: dynamic	Warning flag set — review decay params
flags[].content	flag	First flag used; alternates preserved
hints	hints	Cost and content mapped directly
files	files	Extracted from CTFd ZIP structure
tags	tags	Direct map

11.5 CLI Import

```
feralctf import challenges.json
feralctf import challenges.json --attachments ./files/ --overwrite
feralctf import challenges.json --dry-run
```

Recommended workflow: prepare JSON bundle, validate with --dry-run, then import. Competition can be started immediately from the admin panel.

---

12. UI Reference — Platform Mockup

The following screenshots show the FeralCTF web interface as designed for the initial release. The platform ships with a dark terminal aesthetic by default, using a monospace font throughout for maximum hacker atmosphere. All views are rendered from embedded assets — no CDN or external resources required.

12.1 Challenges View

The main competition interface. Challenges are displayed as cards in a responsive grid, color-coded by category with difficulty indicators, solve counts, and point values. Solved challenges are marked with a corner flag. The category filter bar and search box allow rapid navigation across large challenge sets.

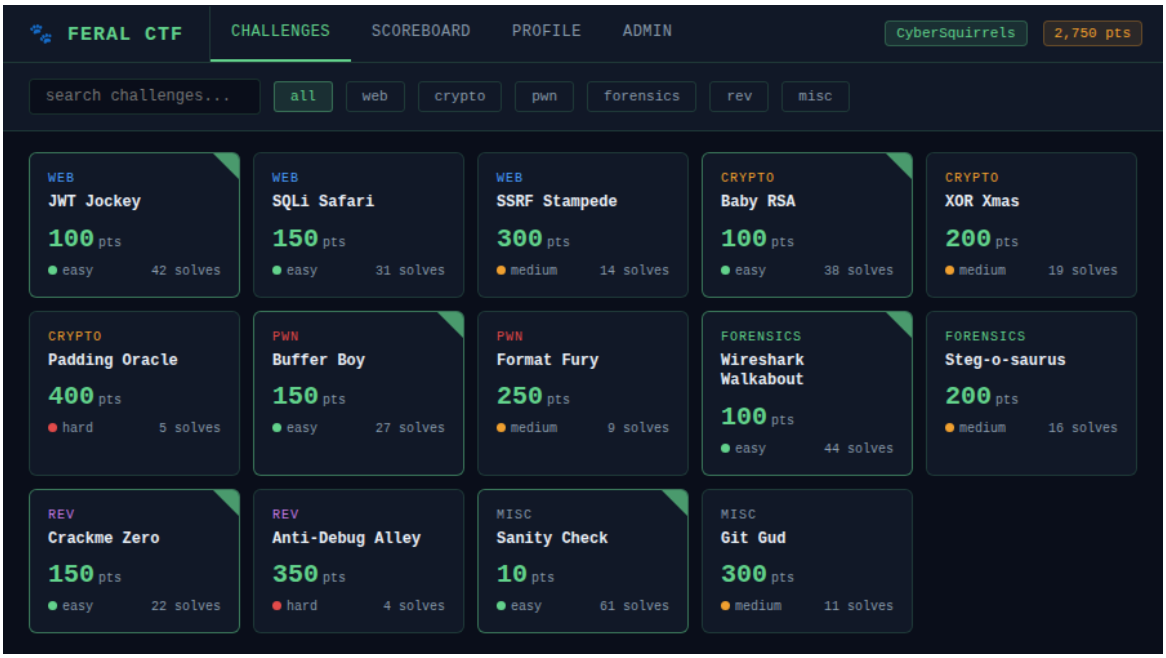


Figure 12.1 — Challenges view. 14 challenges across 6 categories. Green corner flags mark solved challenges.

12.2 Live Scoreboard

Real-time scoreboard updated via WebSocket on every accepted flag submission. Teams are ranked by score with tiebreaking by last solve time. Progress bars give at-a-glance relative standing. The current team row is highlighted.

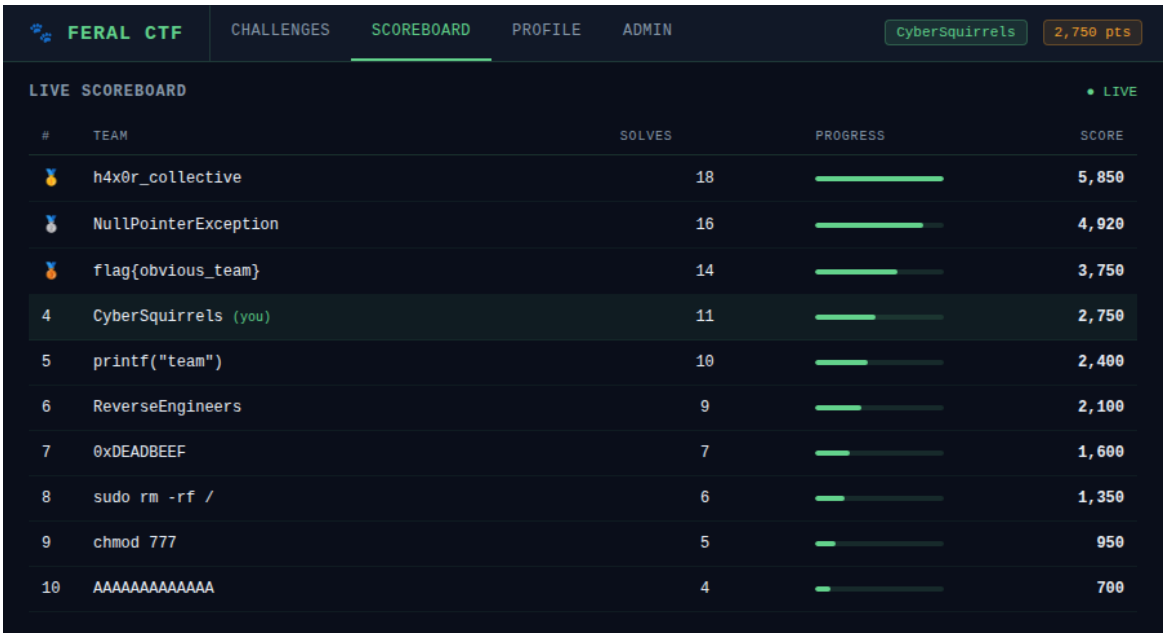


Figure 12.2 — Live scoreboard. Current team highlighted in row 4. Updates in real-time via WebSocket.

12.3 Admin Panel

The admin panel provides a competition overview with live submission stats, full challenge and user management, and competition controls. The overview tab shows a real-time submission log with correct/incorrect indicators.

Figure 12.3 — Admin panel overview. Live submission feed showing team, challenge, and result status.

12.4 Mascot Gallery

Four mascot candidates, all featuring the core FeralCTF motif: a cybernetic squirrel chewing through fiber optic cable in a neon-lit cyberpunk cityscape.

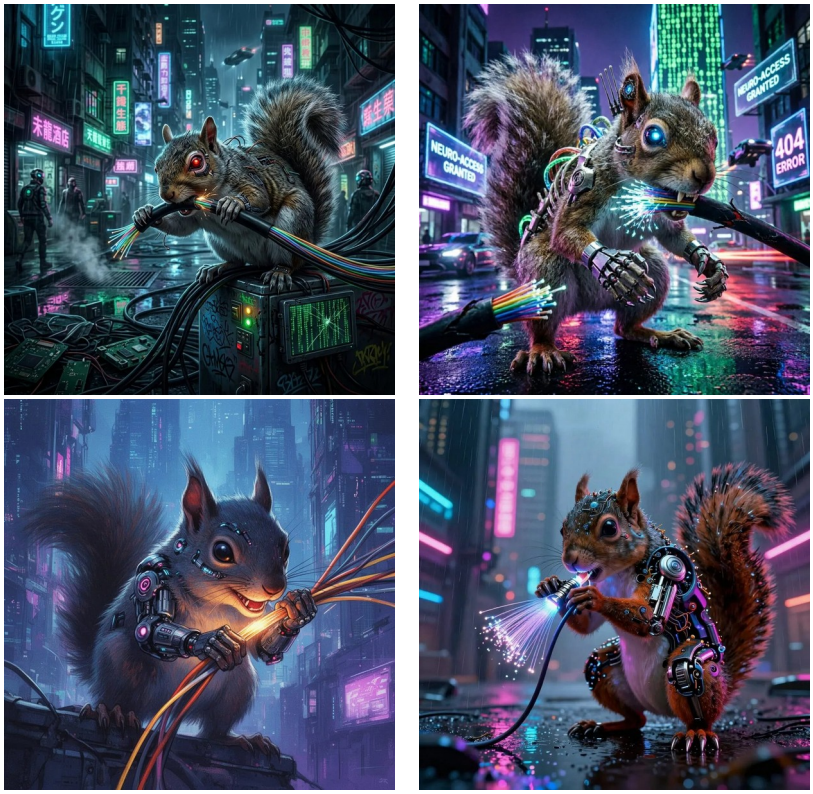


Figure 12.2 — All four mascot candidates. Top-left: photorealistic dark alley. Top-right: photorealistic with neon signage. Bottom-left: illustrated style (recommended). Bottom-right: russet photorealistic in rain.

12.3 UI Design Principles

Element	Design Decision	Rationale
Color scheme	Dark terminal (#0a0e1a background, #63d28c accent)	Matches hacker aesthetic; easy on eyes during long competitions
Font	Courier New / monospace throughout	Consistent with terminal aesthetic; readable at all sizes
Challenge cards	Compact grid with category color coding	Fast scanning across 20-50 challenges
Scoreboard	Live-updating table with team progress bars	WebSocket push; no manual refresh needed
Flag submission	Inline modal — no page navigation	Reduces friction; keeps challenge context visible
Admin panel	Split sidebar + content layout	Mirrors familiar admin UI patterns; efficient for ops under pressure
Mobile support	Responsive grid, collapsible sidebar	On-site competitions often use phones

13. Future Roadmap

Feature	Priority	Notes
CTFd challenge import (full compat)	P1	Covered in Section 11.4
Discord OAuth login	P1	Common in club CTFs
ARM64 binary (Raspberry Pi / AWS Graviton)	P1	Via cross-compilation
Per-challenge Docker spawner (netcat services)	P2	Optional module, not in core binary
TOTP / 2FA for admin accounts	P2	totp-rs crate
CSV/Excel scoreboard export	P2	For awards and post-event records
Graph-based challenge unlock dependencies	P2	Visual tree in admin panel
Writeup submission system	P3	Post-competition learning
CTFtime.org API integration	P3	Auto-publish results
Multi-instance SQLite replication	P3	litestream / Fly.io LiteFS

FeralCTF — Built by defenders, for defenders.